

CS 182 — Deep Neural Networks

Fall 2025

Contents

Lecture 20: In-context learning, prompting and PEFT: Soft-prompting and LoRA	5
20.1 Recap of Large Language Model Mechanics	5
20.1.1 Pre-training GPT-Style Models	5
20.1.2 Inference: Autoregressive Generation	6
20.2 A Brief History of Prompting and In-Context Learning	7
20.3 Instruction Following and Supervised Fine-Tuning (SFT)	8
20.3.1 The Rise of Instruction Tuning	8
20.3.2 Mechanism: Supervised Fine-Tuning (SFT)	8
20.3.3 The Democratization of SFT: The Llama/Alpaca Story (2023)	9
20.4 Parameter-Efficient Fine-Tuning (PEFT)	9
20.4.1 The Spectrum of Model Adaptation	9
20.4.2 Soft Prompting (Prompt Tuning)	10
20.4.3 Prefix Tuning	10
Lecture 21: Finish PEFT and start Transfer Learning	12
21.1 Recap: Fine-Tuning and Gradient Flow	12
21.2 A Spectrum of Approaches for New Tasks	12
21.2.1 Pure Prompting (0-shot, Few-shot)	12
21.2.2 Full Fine-Tuning	12
21.2.3 Parameter-Efficient Fine-Tuning (PEFT)	13
21.2.4 Why Can PEFT Outperform Full Fine-Tuning?	14
21.3 Low-Rank Adaptation (LoRA): A General PEFT Method	14
21.3.1 The Core Idea	14
21.3.2 Training and Inference	15
21.3.3 Initialization and Optimization: Best Practices	15
21.3.4 Advanced Initialization Strategies for A	16
21.3.5 Implications: Model Arithmetic	16

21.4	Meta-Learning: Learning to Fine-Tune	16
21.4.1	Model-Agnostic Meta-Learning (MAML)	16
Lecture 22:	Meta-Learning	18
22.1	Recap: Adapting Pre-trained Models	18
22.1.1	Practical Tips for Full Fine-Tuning	18
22.2	Meta-Learning: Learning to Fine-Tune	19
22.2.1	Model-Agnostic Meta-Learning (MAML)	19
22.2.2	MAML Variants	20
22.3	Catastrophic Forgetting	21
22.4	Introduction to Generative Models	22
22.4.1	Types of Generation	22
22.4.2	Naive Approaches (That Don't Work)	22
22.4.3	Towards Variational Autoencoders (VAEs)	23
Lecture 23:	Generative Models Part 1	24
23.1	Generative Models	24
23.1.1	Introduction	24
23.1.2	Types of Generation	24
23.1.3	Initial Ideas That Don't Work	24
23.2	Variational Autoencoders (VAEs)	25
23.2.1	Key Ideas	25
23.2.2	VAE Architecture and Loss	26
23.2.3	The KL Divergence	26
23.2.4	Making Sampling Differentiable: The Reparameterization Trick	27
23.2.5	Techniques from VAEs in Other Contexts	27
23.3	Post-Training of Large Language Models	28
23.3.1	Recap: Supervised Fine-Tuning (SFT)	28
23.3.2	Improving Problem-Solving Capabilities	28
23.3.3	Advanced Sampling and The Role of RL	29
Lecture 24:	Post-training	30
24.1	Announcements	30

24.2	Recap and Lecture Context	30
24.2.1	Supervised Fine-Tuning (SFT)	30
24.3	Improving Problem-Solving in Models	30
24.4	Test-Time Compute Strategies	31
24.5	Reasoning with Sampling	31
24.5.1	Empirical Observation and Intuition	31
24.5.2	Goal: Sample from a Sharpened Distribution	32
24.5.3	Is This Just Low-Temperature Sampling?	32
24.5.4	MCMC for Sampling	33
24.5.5	The Power Sampling Algorithm	33
24.5.6	Results and Critique	34
24.6	Reinforcement Learning with Verifiable Rewards (RLVR)	34
24.6.1	Scaling Laws for RL	34
24.6.2	The ScaleRL Loss Function	35
Lecture 26: Generative Models Part 2		37
26.1	Announcements and Logistics	37
26.2	Finishing RL Post-Training	37
26.2.1	RL with Verifiable Rewards (RLVR) Recap	38
26.2.2	KL Regularization	38
26.2.3	RL from Human Feedback (RLHF)	39
26.2.4	Direct Preference Optimization (DPO)	40
26.3	Diffusion Models for Generation	41
26.3.1	Motivation: Test-Time Compute	41
26.3.2	The Core Idea of Diffusion	41
26.3.3	The Forward and Reverse Processes	42
26.3.4	Key Design Question: Randomness in the Reverse Path	42
26.3.5	Stochastic Case (DDPM)	42
Lecture 27: Generative Models Part 3		44
27.1	Announcements and Logistics	44
27.2	Introduction to Diffusion Models	44

27.2.1	Recap: The Challenge of Generation	45
27.2.2	The Core Idea of Diffusion	45
27.3	Stochastic Case: Denoising Diffusion Probabilistic Models (DDPM)	45
27.3.1	Continuous-Time Perspective	46
27.3.2	The Reverse Process	46
27.3.3	Argument for the Gaussian Reverse Process	46
27.3.4	Training and Generation	47
27.3.5	An Improved Training Objective	48
27.4	Deterministic Case: Denoising Diffusion Implicit Models (DDIM)	48
27.4.1	Intuition: A Simple Gaussian Example	49
27.4.2	Guess-and-Verify Strategy for the Deterministic Step	49
27.4.3	DDIM Sampling and the ODE Perspective	50

Lecture 20: In-context learning, prompting and PEFT: Soft-prompting and LoRA

20.1 Recap of Large Language Model Mechanics

20.1.1 Pre-training GPT-Style Models

The pre-training process for Generative Pre-trained Transformer (GPT) style models is foundational. The objective is to learn general patterns and knowledge from a vast corpus of text data.

Architecture Overview The model architecture can be broken down into several key components, all of which contain learnable parameters:

1. **Tokenizer:** Converts raw text into a sequence of discrete tokens from a fixed vocabulary (e.g., 32,000 tokens).
2. **Embedder:** A learnable lookup table (a linear layer) that maps each discrete token ID to a high-dimensional continuous vector (the embedding). For example, it might convert a 32,000-dimensional one-hot vector into a 4,096-dimensional embedding.
3. **Transformer Layers:** A stack of identical blocks, each typically containing:
 - Causal Self-Attention mechanisms.
 - Multi-Layer Perceptrons (MLPs).
 - Residual connections and layer normalization.
 - Variations can include State Space Models (SSMs) or Mixture of Experts (MoE) layers.

A critical feature is that the weights of these layers are **shared across all time steps** (positions in the sequence). Interactions across time are handled by the attention mechanism or by state-carrying components in SSMs.

4. **Dembedder** (or Unembedder): A final linear layer that projects the output embeddings from the last transformer layer back into the vocabulary space. It maps the 4,096-dimensional vector back to a 32,000-dimensional vector of logits (scores).
5. **Softmax:** Converts the logits into a probability distribution over the entire vocabulary for the next token.

Training Objective: Next Token Prediction The model is trained on the task of predicting the next token in a sequence.

- **Loss Function:** The standard loss function is the **cross-entropy loss**, which compares the model's predicted probability distribution with the actual next token (the ground truth).

- **Teacher Forcing:** This is the training strategy used. The model is fed the entire ground-truth sequence as input, and at each position t , it is trained to predict the token at position $t + 1$. The model's own predictions are not fed back during training; it is always "forced" to condition on the correct preceding sequence. This allows for massive parallelization during training.
- **Example:** Given the start of the poem "Jabberwocky": *'Twas brillig, and the slithy toves...*
 - After processing the 'start' token, the model's target is "'Twas'.
 - After processing 'start' 'Twas', the target is 'brillig'.
 - After processing 'start' 'Twas brillig', the target is ','.
 - ...and so on, until the end of the document, where the target is an 'end-of-sequence' token.

The quality and mixture of the pre-training data are crucial. Recent work, such as the paper "Textbooks Are All You Need," has shown that high-quality, and even synthetically generated, data can significantly improve model performance.

20.1.2 Inference: Autoregressive Generation

At inference time, the model is used to generate new text in an autoregressive (autocomplete) fashion.

1. **Prompt:** The process begins with a user-provided sequence of tokens called a **prompt**.
2. **Context Processing:** The model processes the entire prompt in one forward pass to compute the internal states (i.e., the key-value cache for all attention layers). No sampling occurs during this phase.
3. **First Token Generation:** Using the state computed from the last token of the prompt, the model generates a probability distribution for the next token.
4. **Sampling:** A token is chosen from this distribution using a sampling strategy (e.g., top-k, nucleus sampling). The **temperature** parameter of the softmax function is a key knob here.

$$P(token_i) = \frac{e^{\text{logit}_i/T}}{\sum_j e^{\text{logit}_j/T}}$$

- A high temperature ($T > 1$) makes the distribution more uniform, leading to more random and creative outputs.
 - A low temperature ($T \rightarrow 0$) makes the distribution sharper, approaching a greedy selection of the most likely token (argmax).
5. **Iteration:** The sampled token is appended to the sequence, and the model performs another forward pass (incrementally, using the cached state) to generate the next token. This loop continues until a special 'stop' token is generated or a maximum length is reached.

20.2 A Brief History of Prompting and In-Context Learning

The way we interact with and utilize large language models has evolved significantly.

- **2017: Transformer:** The "Attention Is All You Need" paper introduced the transformer architecture, laying the groundwork for modern LLMs.
- **2018: BERT and GPT-1:**
 - **BERT** (encoder-only) revolutionized NLP by showing that pre-trained models could produce powerful contextual embeddings, which could be fine-tuned for downstream tasks, leading to state-of-the-art results across the board.
 - **GPT-1** (decoder-only) demonstrated that next-token prediction pre-training also produced high-quality embeddings, positioning it as an alternative to BERT for feature extraction.
- **2019: GPT-2:** By significantly scaling up the model size, GPT-2 demonstrated surprising **emergent abilities**.
 - **Coherent Text Generation:** It could generate shockingly natural prose, poetry, and stories that maintained style and context.
 - **Zero-Shot Task Performance:** The model could perform tasks it was not explicitly trained for, simply by being prompted correctly. This showed it had absorbed factual knowledge.
 - * **Example Prompt:** 'The capital of France is'
 - * **Completion:** 'Paris'
- **2020: GPT-3:** The paper "Language Models are Few-Shot Learners" introduced the concept of **In-Context Learning (ICL)**.
 - **Concept:** By providing a few examples of a task within the prompt itself, the model can learn to perform the task for a new input without any weight updates.
 - **Example:** The model can infer the desired relationship (gemstones vs. fruits) from the examples.

Prompt 1:
red: Ruby; blue:

Completion:
Sapphire

Prompt 2:
red: raspberry; blue:

Completion:

blueberry

- **Significance:** ICL allows solving ML problems with just a few examples, no gradient descent, and no fine-tuning, provided the examples fit within the model’s context window. This sparked the field of “prompt engineering.”

20.3 Instruction Following and Supervised Fine-Tuning (SFT)

While ICL is powerful, a natural next step was to make models more directly useful by having them follow explicit instructions.

20.3.1 The Rise of Instruction Tuning

- Pre-trained models like GPT-3, when prompted with an instruction like “Write me a poem about a cat,” would often complete the instruction rather than execute it (e.g., generating “...and make it rhyme.”).
- **InstructGPT (2021):** This pivotal paper from OpenAI showed how to align a model to follow user instructions. It used a combination of techniques, most notably **Reinforcement Learning from Human Feedback (RLHF)**, to train a model to be a helpful and harmless assistant. This research directly led to the development of ChatGPT.

20.3.2 Mechanism: Supervised Fine-Tuning (SFT)

A core component of instruction tuning, and a simpler alternative to full RLHF, is Supervised Fine-Tuning. SFT adapts a pre-trained model to the instruction-following format.

- **Data:** SFT requires a dataset of high-quality instruction-response pairs. (e.g., {prompt: “Translate ‘hello’ to French”, completion: “bonjour”}).
- **Process:** The training process is similar to pre-training but with a crucial difference in how the loss is calculated.
 1. The model is given the instruction (prompt) as input.
 2. It is then trained to generate the desired response using teacher forcing.
 3. **Masked Loss:** The cross-entropy loss is calculated **only on the response tokens**. No loss is backpropagated from the prompt tokens.
- **Why Mask the Loss?:** The goal is to teach the model how to *respond* to instructions, not how to *generate* them. Training on the entire sequence could lead to confusing behaviors, like the model asking a follow-up question instead of providing an answer.

20.3.3 The Democratization of SFT: The Llama/Alpaca Story (2023)

Initially, instruction tuning was seen as an expensive process requiring massive human annotation effort, accessible only to large labs.

- **LLaMA Release:** Meta released the LLaMA family of models, providing a powerful open-source pre-trained base model.
- **Alpaca (Stanford):** Researchers hypothesized that the key to instruction following was not the complex RLHF process, but simply having enough high-quality examples.
 - They created a dataset by scraping 52,000 instruction-response pairs generated by OpenAI’s powerful models (e.g., GPT-3.5).
 - They performed SFT on the base LLaMA model using this synthetic dataset.
 - **Result:** The resulting model, Alpaca, exhibited impressive instruction-following capabilities, demonstrating that SFT on high-quality synthetic data was remarkably effective. This was a paradigm shift.
- **Dolly (Databricks):** A follow-up question was whether the success was due to the instruction format or some latent property (“GPT-ism”) of the synthetic data. Furthermore, using OpenAI’s output to train a competing model raised licensing concerns.
 - Databricks employees created a smaller, fully human-generated dataset of 15,000 instruction-following examples.
 - They fine-tuned an open model on this dataset.
 - **Result:** The model also acquired strong instruction-following abilities, proving that the **distribution of data** was the key. Pre-training corpora lack sufficient instruction-response examples, and SFT remedies this distributional mismatch.

20.4 Parameter-Efficient Fine-Tuning (PEFT)

Full fine-tuning updates all model parameters (billions of them), which is computationally expensive and requires storing a full model copy for each task. PEFT methods offer a compromise, achieving strong performance by tuning only a tiny fraction of the parameters.

20.4.1 The Spectrum of Model Adaptation

We can think of methods for adapting a pre-trained model on a spectrum:

0. **Feature Extraction (Linear Probing):** Freeze the entire model and train a small prediction head (e.g., a linear classifier) on top of its embeddings. (0% of parameters tuned).
1. **Full Fine-Tuning:** Update all weights in the model via SFT. (100% of parameters tuned).
2. **Pure Prompting / In-Context Learning:** Update no weights at all; all task specification happens in the prompt.

PEFT methods explore the space between (0) and (1).

20.4.2 Soft Prompting (Prompt Tuning)

Instead of manually engineering a discrete text prompt, soft prompting learns an optimal prompt directly in the continuous embedding space.

- **Problem:** Gradients cannot flow back to discrete tokens to optimize a prompt.
- **Solution:** Freeze the entire pre-trained model and learn a sequence of continuous embedding vectors (a "soft prompt") that are prepended to the input.
- **Mechanism:**
 1. A fixed number of "special prompt tokens" are designated (e.g., 100).
 2. These tokens are associated with learnable embedding vectors, which are the **only trainable parameters**.
 3. For each training example, these learnable prompt embeddings are prepended to the (frozen) embeddings of the actual input.
 4. The model processes this combined sequence, and the loss on the response is backpropagated to update *only* the soft prompt embeddings.
- **Parameter Count:** For a model with embedding dimension D_{emb} and a soft prompt of length L_p , the number of trainable parameters is just $L_p \times D_{emb}$.
 - **Example:** For an 8B parameter model with $D_{emb} = 4096$ and $L_p = 100$, we only train $100 \times 4096 \approx 400,000$ parameters, which is $\sim 0.005\%$ of the total.
- **Performance:** Empirically, as the base model size increases, the performance of prompt tuning approaches that of full fine-tuning.

20.4.3 Prefix Tuning

Prefix tuning is a more expressive extension of soft prompting. The insight is that the prompt influences downstream computations via the key-value (KV) states in the attention layers. Prefix tuning learns these intermediate states directly.

- **Mechanism:**
 1. Freeze the entire pre-trained model.
 2. For each transformer layer, introduce a small set of learnable "prefix" key and value vectors.
 3. During the forward pass, for each layer, these learnable KV vectors are prepended to the actual keys and values computed from the input sequence.
 4. The attention mechanism then attends over this combined set of keys and values.

5. The loss is backpropagated to update only these prefix KV vectors across all layers.

- **Parameter Count:** For a model with N_{layers} , prefix length L_p , and model dimension D_{model} :
 - Trainable parameters = $N_{layers} \times L_p \times (D_{key} + D_{value})$. Since keys and values typically span the model dimension, this is roughly $N_{layers} \times L_p \times 2 \times D_{model}$.
 - **Example:** For a 32-layer model with $D_{model} = 4096$ and $L_p = 100$, we train $32 \times 100 \times 2 \times 4096 \approx 26$ million parameters. This is more than prompt tuning but still a tiny fraction ($<0.5\%$) of a full fine-tune.

Lecture 21: Finish PEFT and start Transfer Learning

21.1 Recap: Fine-Tuning and Gradient Flow

When fine-tuning a transformer model for a task like question-answering, a common setup involves providing a prompt (the question) and training the model to generate a specific answer. The loss, typically a cross-entropy loss, is calculated only on the generated answer tokens.

A key point of clarification is the flow of gradients. During backpropagation, the gradient originates from the loss on the answer part. However, due to the nature of the attention mechanism, which connects all tokens to all previous tokens, the gradient flows "back in time" to all preceding parts of the sequence, including the prompt/question tokens.

- The attention blocks create computational paths from the answer tokens back to the question tokens.
- Therefore, even though there is no explicit loss computed on the question part, its representations and the parameters that produced them are updated during fine-tuning. This allows the model to learn how to better process prompts for the given task.

21.2 A Spectrum of Approaches for New Tasks

Given a pre-trained foundation model, there are several ways to adapt it for a new task. These methods exist on a spectrum of how much the model's parameters are changed.

21.2.1 Pure Prompting (0-shot, Few-shot)

This approach requires no gradient updates. It leverages the model's in-context learning capabilities.

- **Pure In-Context Learning:** Provide the model with a few examples (demonstrations) of the task within the prompt itself, followed by the new query.
- **Instruction Tuning:** For models that have been instruction-tuned (like InstructGPT), you can provide a combination of natural language instructions and examples.
- **Prompt Optimization:** The process of finding the optimal prompt (either instructions or examples) can be automated. This involves using a validation set to score different prompts and employing optimization techniques to search the discrete space of tokens for the best-performing prompt.

21.2.2 Full Fine-Tuning

In full fine-tuning, all (or a substantial portion) of the pre-trained model's weights are unfrozen and updated via gradient descent on the new task's data. This is parameter-intensive but often yields high performance.

21.2.3 Parameter-Efficient Fine-Tuning (PEFT)

PEFT methods aim to match the performance of full fine-tuning while only updating a very small number of parameters. This significantly reduces computational and storage costs.

Soft Prompting (Prompt Tuning) Instead of finding optimal "hard" prompts (sequences of discrete tokens), we introduce a "soft prompt" which is a sequence of continuous embedding vectors.

- **Mechanism:** A sequence of learnable embedding vectors is prepended to the input sequence (before the question). These vectors are not tied to any specific tokens in the vocabulary.
- **Parameters:** The only learnable parameters are these new embedding vectors. The entire pre-trained model remains frozen.
- **Gradient Flow:** Gradients from the loss on the answer flow back through the frozen model and are used exclusively to update the soft prompt embeddings.
- **Memory:** While the number of learnable parameters is tiny, the memory required for the forward and backward pass is unchanged. You still need to store all the intermediate activations of the large model for backpropagation (or recompute them). PEFT primarily saves memory on the optimizer states (e.g., momentum and variance terms in Adam), which are only needed for the parameters being updated.

Soft Prefixes (Prefix Tuning) This is a more expressive variant of soft prompting. Instead of learning embeddings at the input layer, we directly insert learnable vectors into the intermediate layers of the transformer.

- **Mechanism:** For the prefix tokens, instead of computing their key (K) and value (V) vectors from the input, we treat these K and V vectors themselves as learnable parameters directly in each attention block.
- When the model processes the question and answer tokens, their queries (Q) attend to these learnable K and V pairs from the prefix.
- **Parameters:** The learnable parameters are the K and V vectors for the prefix sequence, across all layers. These parameters are independent for each layer.
- **Gradient Flow:** Gradients flow back from the answer tokens. When they reach the attention computation involving the prefix, they stop at the learnable K and V parameters, as these parameters do not depend on any earlier computations.

PEFT Parameter Counts Example For a typical large model (e.g., Llama, 7-8B parameters) with an embedding dimension of 4096:

- **Soft Prompt:** A prompt of length 100.

$$\text{Params} = \text{length} \times \text{embedding_dim} = 100 \times 4096 \approx 40\text{k}$$

- **Soft Prefix:** A prefix of length 100, assuming 32 layers and K, V vectors also of size 4096.

$$\text{Params} = \text{length} \times \text{layers} \times (\text{dim}_K + \text{dim}_V) = 100 \times 32 \times (4096 + 4096) \approx 26\text{M}$$

Both are substantially smaller than the 7-8 billion parameters of the full model.

21.2.4 Why Can PEFT Outperform Full Fine-Tuning?

Empirically, it has been observed that PEFT methods can sometimes achieve better generalization on the fine-tuning task than a full fine-tune.

- **Regularization/Overfitting:** The classic machine learning explanation is related to the bias-variance tradeoff. A full fine-tune adjusts billions of parameters, giving it immense capacity to overfit to the specifics of a potentially small fine-tuning dataset.
- By severely restricting the number of learnable parameters (e.g., 40k or 26M), PEFT acts as a strong regularizer, forcing the model to find a more general solution and preventing it from fitting noise in the training data. This is analogous to fitting a line (few parameters) being better for generalization than a 500-degree polynomial (many parameters) on a small dataset.

21.3 Low-Rank Adaptation (LoRA): A General PEFT Method

Observing that different PEFT methods work well for different tasks, researchers sought a more general and powerful recipe. This led to LoRA.

21.3.1 The Core Idea

The central hypothesis of LoRA is that the change in weights needed to adapt a pre-trained model to a new task has a low "intrinsic rank." That is, the adaptation can be represented by a low-rank matrix.

- Instead of updating the original weight matrix $W_0 \in \mathbb{R}^{d \times k}$, we keep it frozen.
- We learn a low-rank update matrix ΔW , such that the new effective weight matrix is $W = W_0 + \Delta W$.
- To enforce the low-rank constraint, ΔW is decomposed into the product of two smaller matrices: $\Delta W = BA$, where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$.
- The rank r is a hyperparameter, typically very small (e.g., 4, 8, 16), which is much smaller than d or k .

The forward pass is modified as:

$$h = Wx = (W_0 + BA)x = W_0x + B(Ax) \tag{1}$$

This means we can implement LoRA as a parallel path to the original frozen layer, whose output is added to the original layer's output.

21.3.2 Training and Inference

- **Training:** During training, W_0 is frozen. Only the parameters of A and B are learnable and receive gradient updates.
- **Inference:** After training, the update can be merged back into the original weights: $W_{new} = W_0 + BA$. The resulting model has the exact same architecture and inference latency as the original model. There is no overhead. This allows for creating multiple "adapted" versions of a model by simply swapping out the main weight matrix.

Note: LoRA is typically applied to matrix-like layers (e.g., attention projections, MLP layers) but not to the token embedding/de-embedding layers, which are collections of vectors rather than true linear operators.

21.3.3 Initialization and Optimization: Best Practices

The initialization of A and B is critical for successful LoRA fine-tuning.

1. **Failed Idea: Initialize $A=0$, $B=0$.** This doesn't work because the gradients for both A and B will be zero at the first step, and the model will never learn.
2. **Failed Idea: Initialize A and B with Xavier/Gaussian.** This is also problematic. At the start of fine-tuning, $\Delta W = BA$ would be a random non-zero matrix. This effectively "shakes" the pre-trained model, moving it far from its well-learned initial state and potentially destroying its capabilities before training even begins. The goal is to *adapt*, not to restart from a random point.
3. **Correct Approach: Initialize $B=0$, A with Xavier/Gaussian.** This ensures that at initialization, $\Delta W = BA = 0$. The fine-tuning process starts exactly from the pre-trained model's state. As training progresses, B moves away from zero, and the model adapts smoothly.

Optimization Details:

- **Optimizer:** Use a standard optimizer like AdamW. Mup-style optimizers are generally not recommended by default for LoRA, as the theoretical assumptions may not hold for this specific architecture.
- **Learning Rates:** Use two different learning rates for A and B . Empirically, a larger learning rate for B than for A works best ($\text{lr}_B = \lambda \cdot \text{lr}_A$, with $\lambda \sim 4 - 10$). This can be loosely motivated by Mup principles, as A and B have vastly different fan-ins (k vs r), but the exact Mup scaling doesn't work perfectly because the rank r is often too small for the theoretical guarantees to hold.

21.3.4 Advanced Initialization Strategies for A

While random Gaussian initialization for A works well, more targeted initializations can potentially improve performance.

- **SVD of Gradients:**

1. Compute a batch of gradients for the full weight matrix W_0 on the fine-tuning data. Let this be G_W .
2. Perform Singular Value Decomposition (SVD) on G_W .
3. Initialize the rows of matrix A with the top r right singular vectors of G_W . This aligns the initial updates with the principal directions of the gradient, potentially speeding up convergence.

- **SVD of Weights:**

1. Perform SVD on the pre-trained weight matrix W_0 itself.
2. Initialize the rows of matrix A with the top r right singular vectors of W_0 . This is based on the hypothesis that the most important directions for the new task are related to the most salient directions (principal components) of the original pre-trained weights.

21.3.5 Implications: Model Arithmetic

The success of LoRA, which treats adaptations as simple additions to the weight space, has bolstered the field of model merging. One can take a base model, fine-tune two different LoRAs on it for two different tasks (e.g., Task 1 and Task 2), and then create a new model by adding both adaptations: $W_{new} = W_0 + B_1A_1 + B_2A_2$. This can often yield a model that is capable at both tasks, sometimes with synergistic effects.

21.4 Meta-Learning: Learning to Fine-Tune

The goal of meta-learning is to train a model that is not just good at a task, but is specifically good at being **fine-tuned** for a new set of tasks. We want to optimize the model's initial state to be a better starting point for future adaptation.

21.4.1 Model-Agnostic Meta-Learning (MAML)

MAML provides a general, conceptual framework for solving the meta-learning problem. While not always used in practice today for LLMs, its core ideas are highly influential.

Problem Formulation: The Ingredients To solve the meta-learning problem, we must first define it formally. We need:

1. **A collection of tasks:** We need a training set of tasks. Each task T_i is itself defined by its own training data and a loss function, e.g., $\{(\mathbf{x}_{i,j}, \mathbf{y}_{i,j}), \mathcal{L}_i\}$.
2. **A defined fine-tuning procedure:** We must specify exactly how adaptation will occur. For example: "take 3 gradient steps with an SGD optimizer and a learning rate of 0.01".
3. **A defined evaluation procedure:** We must specify how we measure success after fine-tuning. For example: "evaluate the performance (loss) on a held-out test set for that task".

The MAML Insight and Algorithm The key insight of MAML is to **train like you test** and make the entire process differentiable. The "test" procedure is fine-tuning. Therefore, we should simulate the fine-tuning process during our meta-training and optimize the initial parameters based on post-fine-tuning performance.

The algorithm can be summarized as:

1. Start with a set of initial (meta) parameters, θ . These are the parameters we want to learn.
2. For a batch of tasks $\{T_i\}$ from our collection:
 - (a) For each task T_i , perform a simulated fine-tuning. This is the **inner loop**.
 - Copy the meta-parameters: $\theta'_i \leftarrow \theta$.
 - Take one or more gradient descent steps on the *training data* of task T_i to get updated parameters: $\theta''_i = \theta'_i - \alpha \nabla_{\theta'} \mathcal{L}_{T_i}(\theta'_i)$.
 - (b) Now, evaluate the performance of the fine-tuned parameters θ''_i on the *test data* of task T_i . This gives a meta-loss, $\mathcal{L}_{T_i}(\theta''_i)$.
3. The total meta-loss is the sum of losses over the batch of tasks, $\sum_i \mathcal{L}_{T_i}(\theta''_i)$.
4. This entire process—from θ to θ'_i to θ''_i to the final loss—is one large computation graph. We can compute the gradient of the meta-loss with respect to the original meta-parameters, $\nabla_{\theta} \sum_i \mathcal{L}_{T_i}(\theta''_i)$.
5. Update the meta-parameters θ using this "meta-gradient". This is the **outer loop**.

Essentially, MAML views the fine-tuning process (a sequence of SGD steps) as a recurrent neural network. The initial parameters θ are the initial hidden state, and the gradients are backpropagated through the unrolled steps of the learning process itself to update this initial state. The result is a model θ that is optimized to be just a few gradient steps away from high performance on a wide range of tasks from the target family.

Lecture 22: Meta-Learning

22.1 Recap: Adapting Pre-trained Models

When adapting a pre-trained model to a new task, several approaches are available, ranging from zero-parameter changes to full model updates.

1. **Prompting:** For models that are "promptable" (e.g., instruction-tuned LLMs), one can simply provide a natural language prompt describing the new task. This is a zero-shot or few-shot approach that leverages in-context learning.
 - **Practical Tip:** If you have data for your task, use a prompt optimizer to find the best-performing prompt instead of relying on manual prompt engineering.
2. **Linear Probing (Feature Extraction):** Treat the pre-trained model as a fixed feature extractor.
 - The process involves passing task data through the frozen pre-trained model to get embeddings.
 - A new, simple task-specific "head" (usually a linear classifier or regressor) is trained on top of these embeddings.
 - **Advantages:** Computationally cheap and fast. Requires only task-specific data.
 - **Disadvantages:** May not perform as well as full fine-tuning.
 - **Observation:** Interestingly, for some tasks, linear probing can outperform prompting, even on large models. This suggests the model's internal representations contain the necessary information, even if it cannot be elicited through prompting alone.
3. **Full Fine-Tuning:** Update all (or a significant portion) of the pre-trained model's parameters, along with a new task-specific head, using the new task's data.
 - **Advantages:** Typically yields the highest performance on the new task.
 - **Disadvantages:** Computationally expensive, requiring significant resources for training and hyperparameter tuning. It also carries a risk of overfitting and *catastrophic forgetting* (discussed later).
4. **Parameter-Efficient Fine-Tuning (PEFT):** Methods like LoRA or soft-prompting/prefix-tuning that are intermediate between linear probing and full fine-tuning. They involve training a small number of additional parameters while keeping the base model frozen.

22.1.1 Practical Tips for Full Fine-Tuning

When performing a full fine-tune with a new task-specific head, the initialization strategy is crucial.

- **Bad Practice:** Initializing the new head with random weights. At the start of training, a random head will produce meaningless outputs and send noisy, unhelpful gradients back

into the pre-trained model. This can disrupt the valuable pre-trained weights before any meaningful learning occurs.

- **Better Practice:** Initialize the new head with all zeros. This ensures that in the initial training steps, zero or near-zero gradients are passed back to the pre-trained model, protecting its weights from random perturbations.
- **Best Practice (Two-Stage Tuning):**
 1. **Stage 1 (Linear Probing):** Freeze the pre-trained model's weights and train only the new task-specific head on your task data. This allows the head to learn a reasonable mapping from the model's features to the task outputs.
 2. **Stage 2 (Full Fine-Tune):** Unfreeze the pre-trained model's weights and continue training the entire model (body and head) with a lower learning rate. At this stage, the gradients flowing from the head are much more meaningful, leading to a more stable and effective fine-tuning process.

22.2 Meta-Learning: Learning to Fine-Tune

Meta-learning aims to train a model that is not just good at a task, but is specifically good at being *fine-tuned* for a family of new, unseen tasks. The goal is to find a model initialization that can adapt quickly and effectively.

22.2.1 Model-Agnostic Meta-Learning (MAML)

MAML is a general framework for learning a good parameter initialization. It relies on two key insights:

1. **Train like you will test:** At test time, we will perform gradient descent (fine-tuning) on a new task. Therefore, the meta-training process should itself involve simulating this fine-tuning process.
2. **Learning as an RNN:** The process of performing multiple steps of Stochastic Gradient Descent (SGD) is a sequential computation. It can be viewed as an unrolled Recurrent Neural Network (RNN), where the model's parameters are the "hidden state." Since RNNs are differentiable, we can backpropagate through the entire learning process.

The MAML Algorithm MAML operates with two loops: an inner loop for task-specific adaptation and an outer loop for meta-optimization.

- **Setup:** We need a distribution of tasks. For each task $\mathcal{T}_i$, we have training data D_i (for the inner loop) and held-out test data $\tilde{D}_i$ (for the outer loop).
- **Outer Loop (Meta-Training):**

1. Sample a batch of tasks $\{\mathcal{T}_i\}$ from the task distribution.
2. For each task $\mathcal{T}_i$:
 - (a) **Inner Loop (Task Adaptation):**
 - Create a copy of the current meta-parameters, $\theta'_0 = \theta$.
 - Perform a few steps of gradient descent on the task's training data D_i to find task-specific parameters θ'_k .

$$\theta'_{j+1} = \theta'_j - \alpha \nabla_{\theta'_j} \mathcal{L}_{\mathcal{T}_i}(\theta'_j, D_i)$$

This inner loop is performed for k steps, resulting in final adapted parameters θ'_k .

- (b) **Meta-Gradient Calculation:** Evaluate the performance of the adapted parameters θ'_k on the task's held-out data $\tilde{D}_i$. The loss on this held-out data, $\mathcal{L}_{\mathcal{T}_i}(\theta'_k, \tilde{D}_i)$, is the meta-loss for this task.
 - (c) The gradient of this meta-loss is computed with respect to the initial meta-parameters θ . This involves backpropagating through all k steps of the inner-loop optimization.
3. **Outer Loop Update (Meta-Optimization):** The meta-parameters θ are updated by taking a step in the direction of the averaged meta-gradients from the batch of tasks.

$$\theta \leftarrow \theta - \beta \nabla_{\theta} \sum_{\mathcal{T}_i} \mathcal{L}_{\mathcal{T}_i}(\theta'_k, \tilde{D}_i)$$

Here, α is the inner-loop learning rate and β is the outer-loop (meta) learning rate.

This process is computationally demanding because it requires computing gradients of gradients ("backprop through backprop") and storing the entire computation graph of the inner loop.

22.2.2 MAML Variants

Reptile Reptile is a first-order approximation of MAML that avoids the expensive second-order derivatives.

- **Idea:** Instead of backpropagating through the inner-loop optimization, approximate the meta-gradient as the difference between the final adapted parameters and the initial meta-parameters.
- **Algorithm:**
 1. Sample a task $\mathcal{T}_i$.
 2. Starting with meta-parameters θ , run SGD for k steps on the task data D_i to obtain adapted parameters θ'_k .
 3. Update the meta-parameters by moving them a small fraction of the way towards the adapted parameters:

$$\theta \leftarrow \theta + \beta(\theta'_k - \theta)$$

- **Benefits:** Computationally much cheaper, requires less memory, and allows for many more inner-loop steps. The intuition is that by taking small steps in the direction of task-specific optima, the meta-parameters will drift towards a point that is, on average, close to the optima of all tasks.

Meta-Learning for Linear Probing If the goal is to learn a model that serves as a good feature extractor (i.e., for linear probing), the inner loop can be made much more efficient.

- **Idea:** For linear models, the optimal head can often be found via a closed-form solution or a fast convex optimization algorithm, both of which are differentiable.
- For regression tasks with a squared-error loss, the optimal linear head has a closed-form solution (Normal Equations). We can differentiate through this closed-form calculation to get a gradient on the features, and thus on the feature extractor's weights.
- For classification tasks, problems like training an SVM or logistic regression are convex. They can be solved rapidly with a few steps of an algorithm like Newton's method. These optimization steps are also differentiable.
- **Benefits:** This approach avoids the iterative nature of the SGD inner loop, replacing it with a single, highly stable, and efficient computation. It eliminates issues with exploding/vanishing gradients in the inner loop and is highly parallelizable.

22.3 Catastrophic Forgetting

Catastrophic forgetting is the phenomenon where a model, when fine-tuned on a new task, loses its ability to perform previously learned tasks.

- **The Problem:** This is particularly problematic when the goal of fine-tuning is to add a general capability (e.g., instruction following, safety alignment) without sacrificing the model's vast pre-trained knowledge. Empirically, aligning a model for safety can make it "dumber" on unrelated, safe tasks.
- **Intuition:**
 - **Leaky Balloon Analogy:** Think of the model's capabilities as different parts of an inflatable structure. The training data for a specific capability acts as an air pump, keeping that part inflated. If you stop "pumping" data for one capability and only pump for another, the first one deflates due to factors like weight decay or gradient interference.
 - **Gradient Interference:** The gradients for the new task, while optimizing for that task, introduce effectively random noise into the weight directions responsible for other tasks. Without a counter-balancing signal from the old task data, the model's weights drift, and performance on old tasks degrades.

- **Standard Solution: Data Replay/Mixing** The most common and effective solution is to mix a small amount (e.g., 5-10%) of the original pre-training data (or data from a similar distribution) into the fine-tuning dataset. This ensures that while the model learns the new task, it continuously receives a "refresher" signal that prevents it from forgetting its general pre-trained capabilities.
- **Forgetting and PEFT:** Methods like LoRA are less susceptible to catastrophic forgetting. This is because (1) the vast majority of the model's weights are frozen, constraining how much the model can change, and (2) weight decay is applied to the low-rank update matrices, which encourages them towards zero, effectively preserving the original pre-trained model's behavior.

22.4 Introduction to Generative Models

Generative models aim to learn the underlying probability distribution of a dataset, $p(\mathbf{x})$, to be able to generate new, unseen samples from that distribution.

22.4.1 Types of Generation

- **Unconditional Generation:** Sample $\mathbf{x} \sim p(\mathbf{x})$. The goal is simply to generate a new, valid sample from the data distribution (e.g., "generate an image of a cat").
- **Conditional Generation:** Sample $\mathbf{x} \sim p(\mathbf{x}|c)$. The generation is conditioned on some input c (e.g., "generate an image corresponding to the text 'a photo of an astronaut riding a horse'").

22.4.2 Naive Approaches (That Don't Work)

Inverting a Classifier

- **Idea 1 (Rejection Sampling):** Train a classifier $p(y|\mathbf{x})$. To generate a "cat," sample a random image $\mathbf{x}$ (e.g., from uniform noise) and accept it if the classifier assigns a high probability to the "cat" class. This fails because the space of natural images is astronomically small compared to the space of all possible pixel combinations. The probability of randomly sampling a coherent image is effectively zero.
- **Idea 2 (Gradient Ascent):** Start with a random noise image $\mathbf{x}$ and use gradient ascent to iteratively modify $\mathbf{x}$ to maximize the classifier's output for the desired class, e.g., $\max_{\mathbf{x}} p(y = \text{cat}|\mathbf{x})$. This also fails to produce realistic images, often resulting in adversarial examples or high-frequency noise patterns that exploit the classifier without resembling a real image.

Using a Standard Autoencoder (AE)

- **Idea:** Train an autoencoder consisting of an encoder $E(\mathbf{x}) = \mathbf{z}$ and a decoder $D(\mathbf{z}) = \hat{\mathbf{x}}$. For generation, discard the encoder and feed random vectors $\mathbf{z}$ (e.g., sampled from a Gaussian) into the decoder.
- **Why It Fails - The Manifold Hypothesis:**
 - Natural data (like images) is assumed to lie on a low-dimensional manifold embedded in a very high-dimensional pixel space.
 - The encoder learns to map points from this data manifold in the input space to a corresponding manifold in the latent space $\mathcal{Z}$.
 - The decoder learns the reverse mapping. However, it only ever sees latent vectors $\mathbf{z}$ that come from the encoder—i.e., points on the latent manifold.
 - When we sample a random $\mathbf{z}$ from a simple distribution (like a Gaussian), it will almost certainly fall into a "hole" or an empty region of the latent space, far from the manifold the decoder was trained on.
 - The decoder's behavior in these unseen regions is undefined and typically results in blurry or nonsensical outputs. The AE is not incentivized during training to produce good outputs for these "off-manifold" latent codes.

22.4.3 Towards Variational Autoencoders (VAEs)

To fix the issue with standard AEs for generation, we need to address two key problems: the lack of robustness in the decoder and the unstructured nature of the latent space.

1. **Inject Randomness During Training:** The "train like you test" principle applies. If we want to feed random $\mathbf{z}$ vectors to the decoder at test time, we must introduce randomness into the latent space during training.
2. **Regularize the Latent Space:** We need to force the distribution of encoded vectors from the training data to match a simple, known prior distribution that we can easily sample from (e.g., a standard normal distribution, $\mathcal{N}(0, I)$). This can be achieved by adding a regularization term (a loss) that penalizes divergence from this prior.

The combination of these ideas leads to the Variational Autoencoder, which trains the decoder to be robust to small perturbations in the latent space, effectively "filling in the holes" and learning a smoother mapping from the latent space to the data manifold.

Lecture 23: Generative Models Part 1

23.1 Generative Models

23.1.1 Introduction

Generative modeling aims to create models that can generate new, unseen data samples that appear to be drawn from the same distribution as a given training dataset. Unlike discriminative models (e.g., classifiers) that learn to map inputs to labels ($p(y|x)$), generative models aim to learn the underlying data distribution itself ($p(x)$).

This can be conceptualized as learning to sample from an unknown distribution. The training process involves two main challenges:

1. Devising an approach that allows for sampling from a distribution.
2. Learning a parameterized form of this distribution from training data.

23.1.2 Types of Generation

There are two primary categories of generative tasks:

- **Unconditional Generation:** The model takes a source of randomness (e.g., a random vector) and produces a new example from the learned distribution. There is no input other than the randomness.

$$\text{Randomness} \rightarrow \boxed{\text{Generative Model (G)}} \rightarrow \text{New Example}$$

- **Conditional Generation:** The model takes both a source of randomness and some conditioning information (e.g., a text prompt, a class label) to produce a new example that is consistent with the conditioning. This is the more practically important case.

$$\text{Conditioning} + \text{Randomness} \rightarrow \boxed{\text{Generative Model (G)}} \rightarrow \text{New Example}$$

Examples include text-to-image generation (conditioning is the text prompt) and language models generating text based on a starting prompt.

23.1.3 Initial Ideas That Don't Work

Using a Classifier A naive idea is to leverage a pre-trained classifier.

- **Idea:** To generate an image of a cat, we can use a cat classifier. An image is a "cat image" if the classifier outputs a high score for the "cat" class.
- **Approach:** Start with a random noise image and perform gradient ascent on the pixels of the image to maximize the cat classifier's output score.

- **Problem:** The space of all possible images is vast, while the "manifold" of natural-looking images is extremely thin and low-dimensional. Most images are just random noise. Gradient ascent on a classifier is not constrained to stay on this manifold.
- **Result:** The process finds an image that is adversarial; it maximizes the classifier's confidence but looks like random noise to a human. The model moves the random image to a nearby point in image space that is confidently "cat", but this point is not on the natural image manifold.

Using an Autoencoder An autoencoder learns a compressed latent representation of data, which seems promising for generation.

- **Architecture:** An autoencoder consists of an **encoder** (E) that maps an input x to a low-dimensional latent code z , and a **decoder** (D) that reconstructs the input $\hat{x}$ from z . It's trained to minimize the reconstruction loss, e.g., $\|x - \hat{x}\|^2$.

$$x \rightarrow \boxed{\text{Encoder}} \rightarrow z \rightarrow \boxed{\text{Decoder}} \rightarrow \hat{x}$$

- **Generative Idea:** Train an autoencoder on a dataset (e.g., cat images). Then, discard the encoder and use only the decoder. To generate a new image, sample a random vector z from a simple distribution (e.g., a Gaussian) and pass it through the decoder.
- **Problem:** Similar to the classifier issue, the latent codes z produced by the encoder for the training data also lie on a very low-dimensional manifold within the latent space. A randomly sampled z is almost certain to be far from this manifold. The decoder has never seen such a z during training and will extrapolate poorly.
- **Result:** The output is typically a noisy or blurry, non-sensical image, not a coherent sample from the data distribution.

23.2 Variational Autoencoders (VAEs)

The Variational Autoencoder (VAE) addresses the shortcomings of the naive autoencoder approach by adding a probabilistic spin. The goal is to make the decoder robust to variations in the latent space so that it can handle randomly sampled z vectors.

23.2.1 Key Ideas

The VAE introduces three core components to the autoencoder framework:

1. **Make z random during training:** Instead of the encoder producing a deterministic z , it will produce the parameters of a probability distribution over z . A sample is then drawn from this distribution to be passed to the decoder. This introduces noise and variation, making the decoder more robust.

2. **Add a loss on the distribution of z :** We enforce that the distribution of z produced by the encoder for a given input x should be "close" to a pre-specified prior distribution (e.g., a standard normal distribution, $\mathcal{N}(0, I)$). This encourages the latent space to be well-structured and dense, preventing gaps.
3. **Make this work with SGD:** The sampling step is inherently non-differentiable. A special technique, the "reparameterization trick," is used to allow gradients to flow back through the sampler to the encoder.

23.2.2 VAE Architecture and Loss

The encoder, $q_\phi(z|x)$, takes an input x and outputs the parameters of a distribution for the latent code z . Typically, this is a Gaussian distribution, so the encoder outputs a mean vector μ_x and a covariance matrix Σ_x . A sample z is then drawn from $\mathcal{N}(\mu_x, \Sigma_x)$. This z is fed into the decoder, $p_\theta(x|z)$, which produces the reconstruction $\hat{x}$.

The VAE is trained to optimize a loss function with two components:

1. **Reconstruction Loss:** Encourages the decoder to accurately reconstruct the input. This is typically the Mean Squared Error (MSE) or binary cross-entropy.

$$\mathcal{L}_{\text{recon}} = \|x - \hat{x}\|^2$$

2. **KL Divergence Loss (Regularizer):** Forces the distribution $q_\phi(z|x)$ produced by the encoder to be close to a chosen prior distribution $p(z)$, usually $p(z) = \mathcal{N}(0, I)$. This is measured by the Kullback-Leibler (KL) divergence.

$$\mathcal{L}_{\text{KL}} = D_{\text{KL}}(q_\phi(z|x) \| p(z))$$

The total loss is a weighted sum: $\mathcal{L} = \mathcal{L}_{\text{recon}} + \beta \mathcal{L}_{\text{KL}}$, where β is a hyperparameter that balances the two objectives. There is an inherent tension: the reconstruction loss pushes the model to encode as much information as possible into z , while the KL loss pushes z to be uninformative (close to the prior).

23.2.3 The KL Divergence

The KL divergence measures the difference between two probability distributions. For continuous distributions Q and P , it is defined as:

$$D_{\text{KL}}(Q \| P) = \int q(z) \log \frac{q(z)}{p(z)} dz = \mathbb{E}_{z \sim Q} \left[\log \frac{q(z)}{p(z)} \right]$$

Note that KL divergence is not symmetric ($D_{\text{KL}}(Q \| P) \neq D_{\text{KL}}(P \| Q)$). In the VAE context, we use $D_{\text{KL}}(q_\phi(z|x) \| p(z))$. This form penalizes q for placing probability mass where p has very little, which is desirable because we want the distributions seen during training to resemble the prior we will sample from at generation time.

For two multivariate Gaussian distributions, $q(z) = \mathcal{N}(\mu, \Sigma)$ and $p(z) = \mathcal{N}(0, I)$, the KL divergence has a convenient closed-form solution:

$$D_{KL}(\mathcal{N}(\mu, \Sigma) \parallel \mathcal{N}(0, I)) = \frac{1}{2} (\text{tr}(\Sigma) + \mu^T \mu - k - \log \det(\Sigma))$$

where k is the dimensionality of the latent space. This formula is fully differentiable with respect to μ and Σ .

23.2.4 Making Sampling Differentiable: The Reparameterization Trick

To train the VAE with gradient descent, we must backpropagate through the sampling step $z \sim q_\phi(z|x)$. This is a stochastic node, which blocks gradients. The reparameterization trick reframes the sampling process to isolate the randomness.

Instead of sampling z directly from $\mathcal{N}(\mu, \Sigma)$, we:

1. Sample a random vector ϵ from a fixed, standard normal distribution: $\epsilon \sim \mathcal{N}(0, I)$.
2. Transform this random vector using the learned mean and covariance to get z : $z = \mu + L\epsilon$, where $\Sigma = LL^T$ (the Cholesky decomposition of Σ).

In this formulation, the randomness is an external input (ϵ), and the path from the encoder's outputs (μ, L) to z is deterministic and fully differentiable. Gradients can now flow from the decoder loss, through z , back to the parameters of the encoder. The stochasticity is treated like any other input data in an SGD setting; over many mini-batches, the expectation of the gradient is what we want.

Parameterizing the Covariance: The network must output a valid covariance matrix (symmetric positive semi-definite). A common technique is to have the network output a lower-triangular matrix L and compute the covariance as $\Sigma = LL^T$. An even simpler approach is to assume a diagonal covariance matrix, in which case the network just outputs the diagonal elements (or their logarithms for numerical stability).

23.2.5 Techniques from VAEs in Other Contexts

The ideas developed for VAEs are broadly applicable.

Quantization-Aware Training (QAT)

- **Problem:** To deploy large models on resource-constrained hardware, we often need to use lower-precision arithmetic (e.g., 8-bit integers instead of 32-bit floats). This process of converting high-precision weights to low-precision is called quantization. A naive post-training quantization can significantly degrade performance.
- **Challenge:** The rounding operation at the core of quantization is non-differentiable, so we cannot naively incorporate it into the training loop.

- **Solution (Straight-Through Estimator):** This technique is analogous to the reparameterization trick.
 - **Forward Pass:** The model's weights are quantized (e.g., rounded to the nearest representable value) before being used in computations.
 - **Backward Pass:** During backpropagation, the gradient is passed through the quantization node as if it were an identity function. We pretend the non-differentiable quantization step didn't happen.

This allows the model to learn weights that are robust to the effects of quantization, improving the performance of the final quantized model.

23.3 Post-Training of Large Language Models

After a base language model is pre-trained via next-token prediction, it undergoes several stages of "post-training" to align it with human preferences and improve its problem-solving abilities.

23.3.1 Recap: Supervised Fine-Tuning (SFT)

The first step is often SFT, where the model is fine-tuned on a high-quality dataset of instruction-response pairs. This teaches the model to follow instructions and act as a helpful assistant.

23.3.2 Improving Problem-Solving Capabilities

Simply training on question-answer pairs is often not enough for complex reasoning tasks. Two key dimensions for improvement have emerged:

1. **Test-Time Compute Scaling:** Spending more computational effort during inference to generate a better answer.
2. **Training for Better Compute Utilization:** Training the model specifically to make better use of this increased inference-time compute. This is where methods like Reinforcement Learning from Verifiable Rewards (RLVR) and Reinforcement Learning from Human Feedback (RLHF) come in.

Test-Time Compute Scaling These are methods that improve performance without changing the model's weights.

- **Chain-of-Thought (CoT) Prompting:** Encouraging the model to "think step by step" by including this phrase in the prompt. This causes the model to generate a rationale before the final answer, which often improves accuracy on reasoning tasks.
- **Best-of-N Sampling:** Generate N different responses to the same prompt (e.g., by using a non-zero temperature) and then select the best one. The challenge is the selection criterion when the ground truth answer is unknown:

- **Majority Vote:** If the answer format is constrained (e.g., multiple choice, a single number), pick the answer that appears most frequently among the N samples.
- **Model-as-Critic:** Use another (or the same) LLM to rate or critique the N responses and select the one with the highest score.
- **Highest Probability:** Select the full sequence (response) to which the model assigned the highest cumulative probability during generation.

23.3.3 Advanced Sampling and The Role of RL

Recent research investigates the connection between post-training and sampling.

- **Observation:** Models that have undergone RLHF/RLVR tend to produce samples that were already among the higher-probability sequences of the original, base model. This suggests that RL might be functioning as a sophisticated sampler, shifting probability mass towards pre-existing "good" regions of the output space.
- **Question:** Can we achieve the benefits of RL post-training with just better sampling methods at inference time, without retraining the model?
- **New Approach: Distribution Tilting:** Sample from a modified probability distribution that amplifies the probability of high-likelihood sequences. This can be done by sampling from a distribution proportional to $p(x)^\gamma$, where $p(x)$ is the original model's sequence probability and $\gamma > 1$.
 - This is related to, but distinct from, low-temperature sampling. Temperature affects each token's probability distribution individually, whereas this tilting applies to the entire sequence probability.
 - Sampling from this new (unnormalized) distribution is non-trivial and may require techniques like Markov Chain Monte Carlo (MCMC).
- **Early Results:** Recent work has shown that these advanced sampling techniques alone can significantly boost model performance on reasoning benchmarks, closing a large portion of the gap between the base SFT model and a fully RL-tuned model. This opens up a new avenue of research teasing apart the effects of training versus sampling in aligning LLMs.

Lecture 24: Post-training

24.1 Announcements

- **Course Survey:** Please fill out the course survey. If 75% of the class completes it by Friday, everyone will receive 3% extra credit. This feedback is valuable.
- **Project Peer Reviews:** Draft projects have been submitted. Everyone has been assigned one review. Please complete your review carefully and provide constructive feedback. The deadline is today, so that feedback can be released before the Thanksgiving break. The goal is to help improve everyone's projects for the final submission.

24.2 Recap and Lecture Context

We have covered various architectures and general deep learning concepts. We began discussing generative models, starting with Variational Autoencoders (VAEs). We are now taking a brief interlude to discuss a critical and current topic: **post-training of models and test-time compute**. These concepts will also be relevant when we discuss diffusion models next week.

The context for this discussion is Large Language Models (LLMs). A model is pre-trained on a vast dataset, but this pre-training distribution may not align perfectly with desired real-world applications, such as instruction following. To bridge this gap, post-training techniques are employed.

24.2.1 Supervised Fine-Tuning (SFT)

One of the first steps in post-training is Supervised Fine-Tuning. This is similar to the pre-training objective (cross-entropy loss), but with a crucial difference: the loss is only computed on the *answer* tokens, not the prompt (question) tokens. This is often called a **masked loss**.

Prompt Tokens $\rightarrow$ **Model** $\rightarrow$ *First Response Token, Second Response Token, ...*
(No Loss) (Compute Cross-Entropy Loss on each generated token)

24.3 Improving Problem-Solving in Models

A key advance for LLMs was the concept of "Chain-of-Thought," where models are prompted or trained to show their work. This revealed that having models generate longer, more reasoned responses improves performance. This leads to a central question: **How can we make a model better at solving problems?**

The answer has two parts:

- A) Be willing to spend more computational resources at inference time. This is known as **test-time compute**.
- B) Train the model to be better at this reasoning process.

24.4 Test-Time Compute Strategies

Spending more compute when a query is made can yield better answers from a pre-trained model.

1. **Prompting:** The simplest approach. Prompt the model with phrases like "Think step by step" or "Be careful." This often encourages more detailed and accurate responses.
2. **Repeated Generation (Best-of-N):** Instead of generating one response, generate N responses by sampling from the model. Since sampling introduces randomness, the responses will differ. Then, select the best one using a criterion:
 - **Majority/Plurality Vote:** If the problem has a discrete, verifiable answer (e.g., A, B, C, D), you can take the most frequent answer among the N generations.
 - **Take Highest Probability:** Choose the generated sequence $\vec{x}$ that has the highest probability $P(\vec{x})$ under the model.
 - **Train a Reward Model:** Train a separate model to score or "grade" each of the N responses. Then, select the response with the highest score from this reward model.
3. **Sample Better:** Instead of just sampling and picking, can we modify the sampling process itself to inherently produce better responses?

24.5 Reasoning with Sampling

A recent paper, "Reasoning with Sampling: Your base model is smarter than you think" (Karan & Du, 2023), explores the idea of improving reasoning without further training.

24.5.1 Empirical Observation and Intuition

It was empirically observed that when models were fine-tuned for better reasoning (e.g., with reinforcement learning), the improved responses they generated often had a *higher probability under the original base model* as well.

This leads to the core idea of the paper: Can we bypass the fine-tuning and get better reasoning simply by finding higher-probability sequences from the base model directly?

Intuition: When a human (or a model) makes a reasoning error, they might "talk themselves into a corner." Subsequent steps become difficult and awkward. For an LLM, this translates to a sequence of low-probability token choices. The entire sequence, being a product of these probabilities, will thus have a very low total probability. Conversely, a correct and coherent line of reasoning might correspond to a higher-probability path.

24.5.2 Goal: Sample from a Sharpened Distribution

The goal is to sample from a distribution that puts more mass on high-probability sequences. We can achieve this by modifying the original probability distribution $P(\vec{x})$:

$$P_{\text{desired}}(\vec{x}) \propto P(\vec{x})^\alpha, \quad \text{where } \alpha > 1$$

Since probabilities are less than 1, raising them to a power $\alpha > 1$ suppresses smaller probabilities more than larger ones. The result must be re-normalized to be a valid distribution.

24.5.3 Is This Just Low-Temperature Sampling?

No. Low-temperature sampling and sampling from $P(\vec{x})^\alpha$ are not the same. Let's analyze this mathematically.

To generate a sequence token by token, we need the conditional probability of the next token x_t given the past $x_{0..t-1}$.

The **desired distribution** for x_t requires marginalizing (summing over) all possible future continuations of the sequence:

$$P_{\text{desired}}(x_t|x_{0..t-1}) \propto \sum_{x_{t+1..T}} (P(x_{0..T}))^\alpha = \sum_{x_{t+1..T}} \left(\prod_{i=0}^T P(x_i|x_{<i}) \right)^\alpha$$

The normalization constant would be a sum over all possible choices for $\tilde{x}_t$.

$$P_{\text{desired}}(x_t|x_{0..t-1}) = \frac{\sum_{x_{t+1..T}} (P(x_{0..T}))^\alpha}{\sum_{\tilde{x}_t} \sum_{x_{t+1..T}} (P(x_{0..t-1}, \tilde{x}_t, x_{t+1..T}))^\alpha}$$

Notice that this involves raising a sum of products to a power.

In contrast, **low-temperature sampling** works differently. It sharpens the distribution at each step greedily, without considering the full sequence probability:

$$P_{\text{low-temp}}(x_t|x_{0..t-1}) \propto (P(x_t|x_{0..t-1}))^\alpha$$

This is equivalent to raising each conditional probability term to the power α inside the product:

$$P_{\text{low-temp}}(x_t|x_{0..t-1}) \propto \frac{(P(x_t|x_{0..t-1}))^\alpha \left(\sum_{x_{t+1..T}} \prod_{i=t+1}^T P(x_i|x_{<i}) \right)^\alpha}{\dots}$$

The key difference is that raising a sum to a power is not the same as the sum of powers: $(\sum a_i)^\alpha \neq \sum (a_i)^\alpha$. Therefore, low-temperature sampling is an approximation that does not correctly sample from the desired distribution over full sequences.

24.5.4 MCMC for Sampling

Since computing the normalization constant for P_{desired} is intractable, we need a different approach. **Markov Chain Monte Carlo (MCMC)** is a class of algorithms designed for sampling from a distribution when we only have access to a function proportional to its probability density.

The idea is to construct a Markov chain whose stationary distribution is our target distribution, P_{desired} . By starting from some initial state (sequence) and taking many steps in this chain, the state we are in will eventually be a sample from the stationary distribution.

The "compute" in test-time compute is spent taking these MCMC steps.

24.5.5 The Power Sampling Algorithm

The paper proposes an algorithm that uses MCMC with a block-wise generation strategy to make it more efficient.

1. **Initialization:** Start with a short prefix.
2. **Block-wise Iteration:** Grow the sequence in blocks of size B . For each block $k = 0, 1, \dots$:
 - (a) **Extend:** Initialize the next block by sampling autoregressively from the base model.
 - (b) **MCMC Steps:** For N_{MCMC} steps:
 - i. Pick a random position m within the current total length $(k+1)B$.
 - ii. **Propose a move:** Create a new candidate sequence x' by resampling from position m to the end using the base model's proposal distribution P_{prop} .
 - iii. **Accept/Reject:** Compute the acceptance probability $A(x', x)$ using a Metropolis-Hastings update rule.

$$A(x', x) = \min \left(1, \frac{\pi_k(x') P_{\text{prop}}(x|x')}{\pi_k(x) P_{\text{prop}}(x'|x)} \right)$$

where $\pi_k(x) \propto P(x)^\alpha$ is our unnormalized target distribution. For autoregressive models, this simplifies nicely. The proposal probabilities cancel, leaving:

$$A(x', x) = \min \left(1, \left(\frac{P(x')}{P(x)} \right)^{\alpha-1} \right)$$

This means a move to a higher probability sequence ($P(x') > P(x)$) is **always accepted**. A move to a lower probability sequence is accepted stochastically.

- iv. Accept the new sequence x' with probability $A(x', x)$, otherwise keep the old sequence x .
 - (c) **Fix Prefix:** After N_{MCMC} steps, fix the current sequence up to $(k+1)B$ and move to the next block.
3. **Return:** Output the final generated sequence.

24.5.6 Results and Critique

The paper shows significant performance gains on reasoning benchmarks without any fine-tuning. For example, on MATH500, the base model (Qwen2.5-7B) achieves 49.6% accuracy, while this sampling method boosts it to 74.8%, which is comparable to a model fine-tuned with RL (GRPO) that gets 78.5%.

Critique (What’s missing?): A critical analysis of the paper’s results raises questions about missing comparisons that would provide a fuller picture:

- **Best-of-N Sampling:** The most direct baseline is to generate N samples and pick the one with the highest probability. How does MCMC compare to this simpler method for the same amount of compute?
- **Beam Search:** Another standard method for finding high-probability sequences.
- **Prompt Optimization:** Could a better prompt have elicited similar gains from the base model?
- **Relative Compute Cost:** The comparison should be normalized by the amount of compute used for each method.
- **Error Bars:** To assess statistical significance.

24.6 Reinforcement Learning with Verifiable Rewards (RLVR)

As distinct from Reinforcement Learning from Human Feedback (RLHF), RLVR is a setting where we have an automated, programmatic way to verify if a model’s response is correct. This is common in domains like math (checking the final answer) or code generation (running unit tests). This simpler setting is a good way to understand the mechanics of RL-based fine-tuning.

24.6.1 Scaling Laws for RL

A key paper, "The Art of Scaling Reinforcement Learning for LLMs" (Khatri et al., 2023), investigates how RL performance scales with compute. They find that the pass rate (a measure of performance) often follows a predictable S-shaped curve when plotted against GPU hours on a log scale.

$$R_c = \frac{A - R_0}{1 + (C_{mid}/C)^B} + R_0$$

This allows them to extrapolate the eventual, asymptotic performance of an RL "recipe" from its initial training trajectory. This is crucial for deciding which methods are worth scaling up.

24.6.2 The ScaleRL Loss Function

The paper proposes a recipe called ScaleRL. Its loss function looks complex, but we can break it down.

$$J_{\text{ScaleRL}}(\theta) = \mathbb{E}_{\substack{x \sim D, \\ \{y_i\}_{i=1}^G \sim \pi_{\text{gen}}^\theta(\cdot|x)}} \left[\frac{1}{|G|} \frac{1}{\sum_{g=1}^G |y_g|} \sum_{g=1}^G \sum_{t=1}^{|y_g|} \text{sg}(\min(\rho_{g,t}, \epsilon)) \hat{A}_g^{\text{norm}} \log \pi_{\text{train}}^\theta(y_{g,t} | y_{g,<t}) \right]$$

This loss is only applied if the pass rate for a given prompt x is less than a threshold (e.g., 90%).

Let's unpack the terms:

- $\mathbb{E}[\dots]$: The expectation is taken over prompts x from the dataset and generations (rollouts) y_i from a **generation policy** π_{gen} .
- $\log \pi_{\text{train}}^\theta(\dots)$: This is the log-probability of a token under the current **model being trained**. The gradient of the loss will be taken with respect to this term's parameters θ . This is the standard "log-probability trick" (from the REINFORCE algorithm) that allows gradients to be estimated for non-differentiable rewards.
- $\hat{A}_g^{\text{norm}}$: **Normalized Advantage**. This is the core reward signal. For each generation y_g in a batch of G generations:
 1. Compute its reward r_g (e.g., 1 if correct, 0 if incorrect).
 2. Compute the mean $\bar{r}$ and standard deviation σ_r of rewards across the batch.
 3. The normalized advantage is $\hat{A}_g^{\text{norm}} = (r_g - \bar{r}) / \sigma_r$.

This normalization centers the rewards, so correct answers get a positive signal (increasing their probability) and incorrect answers get a negative signal (decreasing their probability).

- $\rho_{g,t}$: **Importance Sampling Ratio**. A problem arises because the generations y_g are sampled from π_{gen} , but the gradient is applied to π_{train} . If these policies are different (e.g., π_{gen} is a slightly older, "stale" version of the model), the expectation is mismatched. Importance sampling corrects for this by re-weighting each sample:

$$\rho_{g,t} = \frac{\pi_{\text{train}}^\theta(y_{g,t} | y_{g,<t})}{\pi_{\text{gen}}^\theta(y_{g,t} | y_{g,<t})}$$

This ratio accounts for how much more or less likely a given token is under the current model versus the model that generated it.

- $\min(\rho_{g,t}, \epsilon)$: **Clipping**. The importance sampling ratio is clipped from below by a small constant ϵ . This prevents the weight of a sample from becoming too small if it's very unlikely under π_{train} , which would waste the computational effort of that generation.
- $\text{sg}(\dots)$: **Stop Gradient**. This instructs the automatic differentiation framework (e.g., PyTorch) not to backpropagate gradients through the importance sampling term. This term is just a corrective weight, not part of the objective to be optimized via gradient descent. The gradient should only flow into the $\log \pi_{\text{train}}^\theta$ term.

- **Bag-of-Tokens Perspective:** The loss is calculated per-token and averaged over all tokens in all generations, rather than treating each sequence as a single unit.

This carefully constructed loss function provides a stable and scalable recipe for improving models using reinforcement learning on tasks with verifiable rewards.

Lecture 26: Generative Models Part 2

26.1 Announcements and Logistics

- **Course Topics:** This week finishes RL Post-Training and introduces Diffusion Models, the final topic for the course. A new homework on diffusion models will be released.
- **Survey Extra Credit:** The class survey participation reached 40%, below the 75% target. However, as a thank you, students who completed the survey will receive 1.5% extra credit (prorated from the originally proposed 3%).
- **Final Project Deadlines:**
 - **Final Project Report:** Deadline extended to Sunday, December 7th.
 - **Poster Session:** Tuesday, December 9th, during class time in the Wozniak Lounge. This is a required event. Teams should be present at their posters, especially when staff members visit.
 - **Peer Reviews:** You will be assigned peer reviews of other final reports. These are due by Thursday, December 11th. The feedback is crucial for the final submission.
 - **Final Final Report:** Due Sunday, December 14th. This is the latest possible date before final exam week.
- **Final Exam:**
 - **Date and Time:** Wednesday, December 17th, at 8 AM. Room details to be announced.
 - **Format:** 3 hours long. No notes, no clarifications will be given during the exam. All necessary information will be provided on the exam paper itself.
 - **Preparation:** The best way to prepare is to have internalized the material from homeworks and discussions throughout the semester. The final exam from the previous semester will be posted, but most problem types will have been seen on homeworks.
- **Participation Grading:** A self-grading form will be released. It will involve uploading a picture of yourself and indicating which discussions you attended. For ED participation credit, you can attach screenshots of your posts. Special participation reports should be uploaded to the designated Gradescope assignments.

26.2 Finishing RL Post-Training

We recall the basic pipeline for large language models:

1. **Pre-training:** Unsupervised learning on a massive corpus of text.
2. **Supervised Fine-Tuning (SFT):** Fine-tuning the model on instruction-response pairs to teach it to follow instructions. This typically involves maximizing the log-probability of the desired response tokens using a cross-entropy loss (often with masking so there is no loss on the prompt tokens).

3. **Post-training:** Further training to make the model better at solving problems, often using Reinforcement Learning (RL) techniques. This phase focuses on improving quality, alignment, and helpfulness, and often involves spending more compute at generation time.

26.2.1 RL with Verifiable Rewards (RLVR) Recap

RLVR is used when we have an automated way (an "autograder") to score the quality of a model's generation for a given prompt. The core idea is to generate many candidate responses, score them, and then update the model to increase the probability of high-scoring responses and decrease the probability of low-scoring ones.

The **ScaleRL** paper proposes the following objective function:

$$J_{\text{ScaleRL}}(\theta) = \mathbb{E}_{\substack{x \sim D_x \\ \{y_i\}_{i=1}^G \sim \pi_{\text{gen}}^\theta(\cdot|x)}} \left[\sum_{i=1}^G \sum_{t=1}^{|y_i|} \min(\rho_{i,t}, \epsilon) \hat{A}_i^{\text{norm}} \log \pi_{\text{train}}^\theta(y_{i,t}|y_{i,<t}) \right] \quad (2)$$

Let's unpack the components:

- $\pi_{\text{train}}^\theta$ is the policy (model) we are currently training. π_{gen}^θ is the policy used to generate the responses. In practice, these are different due to engineering constraints (generation servers lag behind the training server). π_{gen}^θ tracks $\pi_{\text{train}}^\theta$ with a delay.
- $\hat{A}_i^{\text{norm}}$ is the normalized advantage for generation y_i . It is calculated by taking the reward r_i for generation y_i , de-meaning it across all G generations for the same prompt, and normalizing by the standard deviation. This tells us how much better or worse a generation is compared to the average.

$$\hat{A}_i = r_i - \text{mean}(\{r_j\}_{j=1}^G), \quad \hat{A}_i^{\text{norm}} = \frac{\hat{A}_i}{\text{std}(\{r_j\}_{j=1}^G)}$$

- The model takes a "bag of tokens" perspective, where every token in a successful generation is considered "good" and inherits the positive advantage of the whole sequence.
- $\rho_{i,t}$ is the importance sampling ratio. It corrects for the mismatch between the generation policy and the training policy.

$$\rho_{i,t} = \frac{\pi_{\text{train}}^\theta(y_{i,t}|y_{i,<t})}{\pi_{\text{gen}}^\theta(y_{i,t}|y_{i,<t})}$$

The gradient is stopped on the denominator ($\text{sg}(\dots)$), as this is just a correction factor.

- $\min(\rho_{i,t}, \epsilon)$ clips the importance weight. If the training policy assigns a much higher probability to a token than the generation policy did, this ratio can become very large, leading to unstable updates. Clipping prevents this.

26.2.2 KL Regularization

A common technique in RL-based fine-tuning is to add a KL-divergence penalty to the objective. This prevents the model from diverging too far from a reference policy, which helps mitigate catas-

trophic forgetting. The objective becomes finding a policy π that maximizes the expected reward while staying close to a reference policy π_{ref} :

$$\arg \max_{\pi} \mathbb{E}_{\substack{x, \\ y \sim \pi(\cdot|x)}} [r(x, y)] - \beta D_{KL}[\pi(\cdot|x) || \pi_{\text{ref}}(\cdot|x)] \quad (3)$$

where β is a hyperparameter controlling the strength of the regularization.

The solution to this optimization problem has the form:

$$\pi(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) e^{\frac{1}{\beta} r(x,y)} \quad (4)$$

Here, $Z(x)$ is a normalization constant. This solution intuitively states that the optimal policy should be the reference policy, re-weighted to put more probability mass on responses with higher rewards.

26.2.3 RL from Human Feedback (RLHF)

In many cases, we don't have a verifiable, automated reward function. RLHF uses human preferences to guide the model.

Key Challenges & Solutions:

1. **Humans are expensive:** It's not feasible to get human feedback for every single generation during RL training. **Solution:** Use human feedback to train an **AI reward model (RM)**. This RM then acts as a proxy for the human, providing dense rewards during RL.
2. **Humans are bad at giving absolute scores:** People are much better at comparative judgments (e.g., "Response A is better than Response B"). **Solution:** Collect data as binary comparisons and train the reward model to predict which response a human would prefer.

The Classic RLHF Pipeline:

1. Start with a pre-trained, SFT model.
2. Generate multiple responses for a set of prompts.
3. Have humans rank these responses or pick the best one (creating preference pairs).
4. Use this preference data to train or update a reward model.
5. Use the trained reward model in an RL loop (like PPO or RLVR) to fine-tune the language model.
6. This is an iterative process. As the language model drifts, the reward model may become inaccurate (out-of-distribution), requiring a new round of data collection and RM training.

26.2.4 Direct Preference Optimization (DPO)

DPO is a more recent technique that bypasses the need for an explicit reward model. It fine-tunes the language model directly on the preference data.

The Problem with Naive Approaches: A simple idea is to treat the preferred response (y_w , for "winner") as a positive example and the dispreferred response (y_l , for "loser") as a negative one. One might try to maximize $\log \pi_\theta(y_w)$ and minimize $\log \pi_\theta(y_l)$.

- **Result:** This often leads to a "total meltdown." The model's generations become incoherent. By pushing down the probability of the loser response, which is often grammatically correct and reasonable but just slightly worse, the model can learn to be ungrammatical or nonsensical to satisfy the objective.

The DPO Approach: DPO proposes a specific loss function derived from a model of human preferences.

1. **Idea 1 (Baseline):** First, perform SFT on only the "winner" responses. This creates a good starting point, or a **reference policy**, π_{ref} .
2. **Idea 2 (DPO Loss):** Starting from $\pi_\theta = \pi_{\text{ref}}$, minimize the following loss function using SGD on preference pairs (y_w, y_l) :

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim D} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right] \quad (5)$$

where σ is the sigmoid function and β is a temperature parameter.

Intuition for the DPO Loss:

- The core of the loss is the difference between two log-probability ratios. This can be interpreted as the difference in the implicit reward between the winner and loser responses. DPO implicitly defines a reward model:

$$\hat{r}_\theta(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$$

This means "what is good is what the current model likes to generate, relative to the reference model."

- **Why does π_{ref} matter?** Gradients do not flow into π_{ref} . So how does it affect training? The terms involving π_{ref} act as a constant offset inside the sigmoid function. This offset shifts the operating point of the sigmoid. If the current model π_θ is already much better than π_{ref} at preferring y_w over y_l , the argument to the sigmoid will be large and positive, putting it in the saturation region where gradients are small. This effectively tells the model "you've learned enough on this example." It focuses the training effort on pairs where the model's preference does not yet align with the human preference.

- **Gradient View:** The gradient of the DPO loss can be shown to be a weighted version of the naive approach:

$$\nabla_{\theta} \mathcal{L}_{\text{DPO}} \propto -(\sigma(\dots) \nabla_{\theta} \log \pi_{\theta}(y_w|x) - \sigma(\dots) \nabla_{\theta} \log \pi_{\theta}(y_l|x))$$

The sigmoid term acts as a dynamic weighting factor on the updates.

Idea 3 (TR-DPO): A further improvement is to periodically update the reference policy π_{ref} to be the current policy π_{θ} every N training steps (e.g., N=512). This allows the baseline for "good enough" to adapt as the model improves.

26.3 Diffusion Models for Generation

We now switch back to generative modeling, introducing a new paradigm: diffusion models.

26.3.1 Motivation: Test-Time Compute

- Models like VAEs and GANs perform a single forward pass for generation: noise in, sample out. This is very fast.
- Autoregressive models spend more compute at test time, generating a sequence one token at a time. This iterative process allows for more complex generation. This works well for domains with inherent sequentiality (text, audio).
- **Question:** How can we spend more compute at test time for non-sequential data like images?

We recall an idea that *didn't* work: starting with random noise and performing gradient ascent on the output of a classifier (e.g., a "cat" classifier). This fails because the classifier is trained on the manifold of natural images, and random noise is far off this manifold. The gradients in this out-of-distribution space are not meaningful for generating realistic images.

The insight from VAEs was that the decoder must be trained on noisy inputs to learn to map a noisy latent space to clean outputs. Diffusion models take this idea to an extreme.

26.3.2 The Core Idea of Diffusion

Instead of just one level of noise, what if we train the model on a whole spectrum of noise?

1. We start with a **clean desired sample** (e.g., a real image).
2. At the other extreme, we have **total noise** (e.g., a sample from a standard Gaussian).
3. The bridge between these two is a continuum of samples with varying levels of noise. We can generate these by taking a clean sample and progressively adding more and more noise.

The model will be trained on data from this entire continuum. The generative process will then involve starting from total noise and iteratively *removing* noise to arrive at a clean sample.

26.3.3 The Forward and Reverse Processes

- **Forward Process (Diffusion):** This is a fixed process that we define. We start with a data sample $x_0 \sim q(x)$ and gradually add Gaussian noise over a series of T timesteps, producing a sequence of latent variables $x_1, x_2, \dots, x_T$.

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t\mathbf{I})$$

The β_t are small positive constants. After many steps, x_T becomes approximately an isotropic Gaussian $\mathcal{N}(0, \mathbf{I})$. For conceptual simplicity, one can think of this as simply adding noise: $x_t = x_{t-1} + \text{noise}$. The scaling factors $\sqrt{1 - \beta_t}$ are a practical detail to keep the variance bounded.

- **Reverse Process (Generation):** The goal is to learn the reverse process, $p_\theta(x_{t-1}|x_t)$. If we can model this distribution, we can start with a sample $x_T \sim \mathcal{N}(0, \mathbf{I})$ and iteratively sample backwards: $x_{T-1}, x_{T-2}, \dots, x_0$ to generate a new data point.

The analogy is a movie of a drop of dye diffusing in water. The forward process is the movie playing forward (dye spreads out to equilibrium). The reverse process is learning to run the movie backward (randomly distributed dye particles coalesce back into a drop).

26.3.4 Key Design Question: Randomness in the Reverse Path

When we generate by moving backward, does each step need to be random?

1. **Stochastic Sampling (DDPM - Denoising Diffusion Probabilistic Models):** Yes, the backward path is stochastic. This is analogous to simulating the reverse of a physical process.
2. **Deterministic Sampling (DDIM - Denoising Diffusion Implicit Models):** No, the backward path can be deterministic. We will discuss this in the next lecture.

26.3.5 Stochastic Case (DDPM)

To build intuition, consider a simple discrete-state Markov chain with 3 states (A, B, C) that converges to a uniform stationary distribution $(1/3, 1/3, 1/3)$.

- Suppose our "data distribution" is non-stationary, e.g., $p_0(A) = 1/2, p_0(B) = 0, p_0(C) = 1/2$.
- **Forward process:** We run the Markov chain forward. Over time, any initial distribution will converge to the stationary uniform distribution.
- **Reverse process:** If we watch a "movie" of this process in reverse, the transition probabilities are different. For example, after one step, if we find ourselves in state B, we know we must have come from A or C, not B. The reverse process is time-varying; it has a "drift" that pulls the distribution back towards the initial, non-stationary one.

Applying this to the continuous case of images, a key result is that if the forward noising step is a Gaussian, the reverse step is *also* approximately Gaussian, as long as the noise added (β_t) is small.

$$p(x_{t-1}|x_t) \approx \mathcal{N}(x_{t-1}; \boldsymbol{\mu}_\theta(x_t, t), \boldsymbol{\Sigma}_\theta(x_t, t))$$

Usually, the variance $\boldsymbol{\Sigma}_\theta$ is fixed, so the learning task is to train a neural network $\boldsymbol{\mu}_\theta(x_t, t)$ to predict the mean of the distribution for the previous, less noisy state, given the current noisy state x_t and the timestep t . This network is trained to "denoise" the input by one step. The generation process then involves calling this network repeatedly to move from pure noise back to a clean image.

Lecture 27: Generative Models Part 3

27.1 Announcements and Logistics

This is the final lecture of the course. Key reminders and deadlines are as follows:

- **Final Project Report:** The deadline has been extended to Sunday, December 7th.
- **Poster Session:** Tuesday, December 9th.
 - Students are responsible for printing their own posters. It is perfectly acceptable to print slides on regular paper and arrange them together. The focus is on clear communication of your work, not on professional printing quality.
 - It is recommended to bring a printed copy of your report and a laptop to show additional plots, code, or simulations during discussions.
- **Peer Reviews:** Assigned on Monday, December 8th, and due on Thursday, December 11th. Your individual grade for conducting peer reviews will be a multiplicative factor on your group's project grade.
- **Final Report Submission:** After incorporating feedback from peer reviews and the poster session, the final version of the report is due Sunday, December 14th. The final grade is based on the quality of this final report, the poster session presentation, and the quality of your peer reviews.
- **Final Exam:** Wednesday, December 17th, at 8:00 AM.
 - The exam is closed-book: no notes, no questions, and no clarifications will be permitted during the exam.
 - You will only need a pencil and an eraser.
- **Recommended EECS Colloquium:**
 - Wednesday, December 10th, 4-5 PM by Ece Kamar from Microsoft Research, who leads the group releasing models like Phi.
 - The recent colloquium by Bill Dally from NVIDIA is also highly recommended for understanding the interplay between hardware (GPUs) and deep learning architectures.

27.2 Introduction to Diffusion Models

This lecture covers diffusion models for generative tasks. The primary readings are Chapter 18 of the Prince textbook and the paper arXiv:2406.08929, “Step-by-step Diffusion: An elementary tutorial,” which provides a clear and modern perspective.

27.2.1 Recap: The Challenge of Generation

The goal of generative modeling is to sample from the distribution of "natural" data, such as images. This distribution forms a very thin, low-dimensional manifold within the vast space of all possible pixel combinations.

- A naive approach, like starting with random noise and performing gradient ascent on a classifier's score (e.g., "cat score"), fails. It produces noise-like images that maximally activate the classifier but do not resemble natural images.
- The insight from Variational Autoencoders (VAEs) is that the model needs to see noisy versions of the data during training to learn how to navigate the high-dimensional space and move towards the data manifold.

27.2.2 The Core Idea of Diffusion

Diffusion models embrace this idea by systematically corrupting data with noise and then learning to reverse the process.

- **Analogy:** Imagine a drop of red dye (the manifold of natural images) placed in a glass of water (the space of all possible images). Over time, the dye diffuses until it is uniformly spread throughout the water.
- **Forward Process (Diffusion):** This corresponds to gradually adding noise to an image, step-by-step, until it becomes pure, unstructured noise (like the fully diffused dye). This process maps a complex data distribution to a simple, known distribution (e.g., a standard Gaussian).
- **Reverse Process (Generation):** This corresponds to running the diffusion movie in reverse. We start with a sample from the simple noise distribution and learn a process to gradually denoise it, step-by-step, until it becomes a sample from the original data distribution.

The key design question is whether the reverse path needs to be stochastic or if it can be deterministic. We will explore both approaches.

1. **Stochastic Sampling:** Denoising Diffusion Probabilistic Models (DDPM).
2. **Deterministic Sampling:** Denoising Diffusion Implicit Models (DDIM).

27.3 Stochastic Case: Denoising Diffusion Probabilistic Models (DDPM)

The first approach is to simulate the exact reverse of the forward stochastic process.

27.3.1 Continuous-Time Perspective

For clarity, we adopt a continuous-time perspective where the process evolves from time $t = 0$ (clean data) to $t = 1$ (pure noise). The discrete steps are of size Δt .

- Let X_0 be a sample from the data distribution p_0 .
- The forward process adds Gaussian noise at each step. In a small time step Δt , the added noise is $\mathcal{N}(0, \sigma^2 \Delta t I)$. The variance scales with Δt to ensure a well-defined limit. The total variance after time t is $t\sigma^2$.
- We define a sequence of noisy images $X_0 \rightarrow X_{\Delta t} \rightarrow X_{2\Delta t} \rightarrow \dots \rightarrow X_1$.

27.3.2 The Reverse Process

Our goal is to learn the conditional distribution $p(X_{t-\Delta t}|X_t)$.

Key Claim: As long as the data density p_0 is sufficiently smooth and the noise step $\sigma^2 \Delta t$ is small enough, the reverse conditional distribution is also approximately Gaussian:

$$p(X_{t-\Delta t}|X_t = z) \approx \mathcal{N}(\mu_t(z), \sigma^2 \Delta t I) \quad (6)$$

Crucially, the variance of the reverse step is the same as the forward step. The mean $\mu_t(z)$ is a function of the current state z and time t , and this is what we must learn from data using a neural network.

27.3.3 Argument for the Gaussian Reverse Process

We can justify this claim with an argument based on Bayes' rule and Taylor expansions.

1. **Bayes' Rule:** Start by expressing the reverse conditional in terms of the forward conditional:

$$p(x_{t-\Delta t}|x_t) = \frac{p(x_t|x_{t-\Delta t})p_{t-\Delta t}(x_{t-\Delta t})}{p_t(x_t)} \propto p(x_t|x_{t-\Delta t})p_{t-\Delta t}(x_{t-\Delta t})$$

The denominator $p_t(x_t)$ is a normalization constant that does not depend on $x_{t-\Delta t}$.

2. **Log-Probabilities:** Taking the logarithm simplifies the product into a sum:

$$\log p(x_{t-\Delta t}|x_t) = \log p(x_t|x_{t-\Delta t}) + \log p_{t-\Delta t}(x_{t-\Delta t}) + \text{const}$$

3. **Substitute Known Terms:** The forward process is Gaussian, so $\log p(x_t|x_{t-\Delta t}) = -\frac{1}{2\sigma^2 \Delta t} \|x_t - x_{t-\Delta t}\|_2^2 + \text{const}$. For sufficiently small Δt , $p_{t-\Delta t} \approx p_t$, so we can approximate $\log p_{t-\Delta t}(x_{t-\Delta t}) \approx \log p_t(x_{t-\Delta t})$.

$$\log p(x_{t-\Delta t}|x_t) \approx -\frac{1}{2\sigma^2 \Delta t} \|x_t - x_{t-\Delta t}\|_2^2 + \log p_t(x_{t-\Delta t}) + \text{const}$$

4. **Taylor Expand around x_t :** We expand $\log p_t(x_{t-\Delta t})$ around the point x_t :

$$\log p_t(x_{t-\Delta t}) \approx \log p_t(x_t) + \langle \nabla_{x_t} \log p_t(x_t), x_{t-\Delta t} - x_t \rangle + O(\|x_{t-\Delta t} - x_t\|^2)$$

The term $\log p_t(x_t)$ is constant with respect to $x_{t-\Delta t}$ and can be absorbed into the normalization constant. The gradient term $\nabla_{x_t} \log p_t(x_t)$ is known as the **score**.

5. **Complete the Square:** Combining terms, we have an expression that is quadratic in $x_{t-\Delta t}$:

$$\log p(x_{t-\Delta t}|x_t) \approx -\frac{1}{2\sigma^2\Delta t}\|x_t - x_{t-\Delta t}\|_2^2 - \langle \nabla_{x_t} \log p_t(x_t), x_t - x_{t-\Delta t} \rangle + \text{const}$$

This is the classic setup for completing the square. By rearranging terms, we can see this is the log-density of a Gaussian.

$$\begin{aligned} \log p(x_{t-\Delta t}|x_t) &\propto -\frac{1}{2\sigma^2\Delta t} (\|x_t - x_{t-\Delta t}\|_2^2 + 2\sigma^2\Delta t \langle \nabla_{x_t} \log p_t(x_t), x_{t-\Delta t} - x_t \rangle) \\ &\propto -\frac{1}{2\sigma^2\Delta t} \|(x_{t-\Delta t} - x_t) - \sigma^2\Delta t \nabla_{x_t} \log p_t(x_t)\|_2^2 \\ &\propto -\frac{1}{2\sigma^2\Delta t} \|x_{t-\Delta t} - (x_t + \sigma^2\Delta t \nabla_{x_t} \log p_t(x_t))\|_2^2 \end{aligned}$$

This reveals the form of a Gaussian with:

- **Mean:** $\mu_t(x_t) = x_t + \sigma^2\Delta t \nabla_{x_t} \log p_t(x_t)$
- **Variance:** $\sigma^2\Delta t I$

27.3.4 Training and Generation

Generation (DDPM Sampling): With a trained model $f_\theta(x_t, t)$ that approximates the mean $\mu_t(x_t)$, we can generate samples.

1. Start with pure noise: $\hat{x}_1 \sim \mathcal{N}(0, \sigma^2 I)$.
2. Iterate backwards from $t = 1$ down to $t = \Delta t$: For each step, compute the next state by taking a sample from the reverse conditional Gaussian:

$$\hat{x}_{t-\Delta t} = f_\theta(\hat{x}_t, t) + \eta, \quad \text{where } \eta \sim \mathcal{N}(0, \sigma^2\Delta t I)$$

3. The final output is $\hat{x}_0$.

Training: We train the network f_θ using stochastic gradient descent.

1. Sample a clean image x_0 from the training set.
2. Sample a random time t uniformly from $[0, 1]$.
3. Construct a noisy version x_t by adding Gaussian noise with variance $t\sigma^2$ to x_0 .

4. The network $f_\theta(x_t, t)$ is trained to predict the mean of the reverse step. The ideal target is $\mathbb{E}[X_{t-\Delta t}|X_t = x_t]$. We can use $x_{t-\Delta t}$ (generated from x_0) as a target.
5. The loss function is the squared error:

$$L_\theta = \|f_\theta(x_t, t) - x_{t-\Delta t}\|_2^2$$

27.3.5 An Improved Training Objective

Instead of predicting $x_{t-\Delta t}$, we can re-parameterize the prediction target. Note that the mean $\mu_t(x_t)$ can be rewritten as $\mu_t(x_t) = x_t + (\mu_t(x_t) - x_t)$, where $\mu_t(x_t) - x_t$ is the expected negative of the noise added in the last step.

$$\mu_t(x_t) - x_t \text{ is an estimate of } \mathbb{E}[x_{t-\Delta t} - x_t|x_t] \quad (7)$$

The key insight comes from the symmetry of the i.i.d. noise added over time. Given only the final noisy image x_t , there is no way to distinguish the noise added in the last step from noise added in any other step. Therefore, the expectation of the last noise increment is simply the average of all noise increments.

$$\mathbb{E}[x_{t-\Delta t} - x_t|x_t] = \frac{\Delta t}{t} \mathbb{E}[x_0 - x_t|x_t] \quad (8)$$

This implies that learning to predict the last noise step is equivalent to learning to predict the total noise added ($x_t - x_0$), which is in turn equivalent to predicting the original clean image x_0 .

Benefits of Predicting x_0 :

- **Inductive Bias:** Neural network architectures like U-Nets and ConvNets are designed with strong inductive biases for processing and generating natural images, not for generating unstructured noise. Training the network to output something image-like is more effective.
- **Variance Reduction:** The target x_0 is a single, clean image, which is a lower-variance target than a noisy sample, leading to more stable training.

So, a better training objective is to have the network $f_\theta(x_t, t)$ predict an estimate of x_0 , and use the loss $\|f_\theta(x_t, t) - x_0\|_2^2$.

27.4 Deterministic Case: Denoising Diffusion Implicit Models (DDIM)

The DDPM sampling process walks backward through the same joint distribution as the forward process. However, to generate a sample from p_0 , we only need the marginal distributions p_t to be correct at each step. This opens the possibility of finding a different, deterministic path that connects the same sequence of marginals.

27.4.1 Intuition: A Simple Gaussian Example

Consider two Gaussian marginal distributions, $X \sim \mathcal{N}(0, 1)$ and $Y \sim \mathcal{N}(0, 2)$.

- **Stochastic Connection (DDPM-like):** Define $Y = X + \epsilon$, where $\epsilon \sim \mathcal{N}(0, 1)$ is independent of X . The reverse process is stochastic: given Y , the distribution of X is $\mathcal{N}(Y/2, 1/2)$. Sampling X from Y requires adding new noise.
- **Deterministic Connection (DDIM-like):** Define $Y = \sqrt{2}X$. This preserves the marginals. The reverse process is purely deterministic: $X = Y/\sqrt{2}$. No additional randomness is needed.

This example shows that it is possible to connect the same marginals deterministically.

27.4.2 Guess-and-Verify Strategy for the Deterministic Step

We want to find a deterministic update rule. Let's guess a form for the reverse step. We know that $(\mu_t(x_t) - x_t)$ is the predicted direction towards a less noisy state. A natural guess is to move in this direction:

$$\hat{x}_{t-\Delta t} = \hat{x}_t + \lambda(\mu_\theta(\hat{x}_t, t) - \hat{x}_t) \quad (9)$$

The problem is to find the correct scaling factor λ . We use the strategy of solving for λ in the simplest possible case and hoping the solution generalizes.

The Easiest Example: Assume the data distribution is a single point at the origin: $p_0 = \delta_0$.

- In this case, we know all marginals exactly: $x_t \sim \mathcal{N}(0, t\sigma^2)$.
- The true conditional mean is also known: $\mathbb{E}[X_{t-\Delta t} | X_t = z] = \frac{t-\Delta t}{t}z$.
- Substitute this into our guessed update rule:

$$\begin{aligned} \hat{x}_{t-\Delta t} &= \hat{x}_t + \lambda \left(\frac{t-\Delta t}{t} \hat{x}_t - \hat{x}_t \right) \\ &= \hat{x}_t \left(1 - \lambda \frac{\Delta t}{t} \right) \end{aligned}$$

Matching Variances: For the marginals to be correct, the variance of our generated $\hat{x}_{t-\Delta t}$ must match the true variance, which is $(t - \Delta t)\sigma^2$.

$$\begin{aligned} \text{Var}(\hat{x}_{t-\Delta t}) &= \text{Var} \left(\hat{x}_t \left(1 - \lambda \frac{\Delta t}{t} \right) \right) \\ (t - \Delta t)\sigma^2 &= \left(1 - \lambda \frac{\Delta t}{t} \right)^2 \text{Var}(\hat{x}_t) \\ (t - \Delta t)\sigma^2 &= \left(1 - \lambda \frac{\Delta t}{t} \right)^2 (t\sigma^2) \end{aligned}$$

Solving this equation for λ yields:

$$\lambda = \frac{t}{\Delta t} \left(1 - \sqrt{\frac{t - \Delta t}{t}} \right) \quad (10)$$

For large t , $\sqrt{1 - \frac{\Delta t}{t}} \approx 1 - \frac{1}{2} \frac{\Delta t}{t}$, so $\lambda \approx \frac{t}{\Delta t} (\frac{1}{2} \frac{\Delta t}{t}) = 1/2$. This means for most of the reverse process, we step about half the predicted distance.

27.4.3 DDIM Sampling and the ODE Perspective

This derived value of λ works in the general case, leading to the DDIM sampling algorithm.

Generation (DDIM Sampling):

1. Start with pure noise: $\hat{x}_1 \sim \mathcal{N}(0, \sigma^2 I)$.
2. Iterate backwards from $t = 1$ down to $t = \Delta t$:

$$\hat{x}_{t-\Delta t} = \hat{x}_t + \lambda(t)(f_\theta(\hat{x}_t, t) - \hat{x}_t)$$

where f_θ is the same network trained for DDPM (predicting x_0), and $\lambda(t)$ is the schedule derived above.

3. The final output is $\hat{x}_0$.

This process can be viewed as solving an Ordinary Differential Equation (ODE). The update rule is a discretization of a continuous flow defined by a velocity field $v_t(x_t)$:

$$\frac{d\hat{x}}{dt} = v_t(\hat{x}_t) \quad (11)$$

The model learns a vector field that flows from any point in space towards the data manifold. The provided figure of a spiral dataset illustrates this: with enough training samples, the learned trajectories (velocity field) correctly flow towards the spiral structure, effectively learning the geometry of the data manifold. This also highlights that the quality of the learned manifold depends on the density of the training data.